

ASSISTENTE-SHI - Xiaozhi AI Assistant

 release no releases or repo not found Licença MIT  stars 0

Português Brasileiro | [中文简体](#) | [English](#)

Sobre o Projeto

ASSISTENTE-SHI é um cliente Python avançado de inteligência artificial (IA) baseado na arquitetura do Xiaozhi. Ele permite interação por voz com IA de forma natural e fluida, oferecendo suporte completo a múltiplas ferramentas MCP, integração IoT, processamento de áudio de alta qualidade e interface gráfica moderna.

Este é um fork personalizado do [py-xiaozhi](#) com melhorias, correções e otimizações específicas.

✦ Características Principais

Funcionalidades de IA

- **Interação por Voz com IA:** Suporte completo a entrada de voz, reconhecimento de fala e resposta inteligente
- **Visão Computacional:** Reconhecimento e processamento de imagens com capacidades multi-modal
- **Despertar Inteligente:** Múltiplas palavras-chave de ativação (configuráveis)
- **Modo de Conversa Contínua:** Experiência de conversa fluida e natural

Modelos de IA e APIs

Configuração de Modelos LLM

O ASSISTENTE-SHI suporta múltiplos modelos de linguagem com fallback automático:

1 GLM-4V (Zhipu API) - PRINCIPAL (com restrições)

 Status: Configurado mas com token inválido
 Modelo: GLM-4V-Plus (Zhipu / 智谱清言)
 API: <https://api.tenclass.net/xiaozhi/vision/explain>
 Custo: ¥0.1 - 0.5 / 1K tokens (aproximadamente R\$ 0.08 - R\$ 0.40)
 Requisitos: Token de autenticação válido

Vantagens Técnicas da API Xiaozhi/GLM-4V:

- ☒ Visão computacional multi-modal integrada
- ☒ Suporte a reconhecimento de caracteres chineses (OCR)
- ☒ Análise de imagens em tempo real
- ☒ Latência baixa (~500ms-1s)

- ☒ Integração nativa com protocolo Xiaozhi
- ☒ Custo relativamente baixo para visão computacional
- ☒ Suporte a contexto visual expandido

Limitações Conhecidas:

- ☒ Token de autenticação inválido/expirado nesta instância
- ☒ Retorna erro 404 em endpoints de visão
- ☒ Requer renovação periódica de credenciais
- ☒ Disponibilidade limitada a regiões específicas

Configuração (config/config.json):

```
{
  "llm": {
    "api": "zhipu",
    "model": "glm-4v-plus",
    "token": "d656b80e-4d21-46d6-a32e-74048bf28a47",
    "base_url": "https://api.tenclass.net/xiaozhi/vision/explain"
  }
}
```

2 LLaVA (Ollama Local) - FALLBACK RECOMENDADO ★

-  Status: Ativo e Funcional (Substituição do GLM-4V)
-  Modelo: LLaVA 1.6 (7B/13B/34B - Configurável)
-  Execução: Local (sem internet necessária)
-  Custo: Gratuito (Open Source)
-  Requisitos: Ollama instalado + 4GB+ VRAM

Vantagens Técnicas do LLaVA + Ollama:

- ☒ **100% Gratuito** - Sem custo recorrente
- ☒ **Privacidade Total** - Execução completamente local
- ☒ **Sem Dependência de Internet** - Funciona offline
- ☒ **Sem Limite de Tokens** - Uso ilimitado
- ☒ **Sem Rate Limiting** - Velocidade consistente
- ☒ **Customizável** - Modelos ajustáveis (7B/13B/34B)
- ☒ **Open Source** - Código aberto e auditável
- ☒ **Visão Computacional** - Análise de imagens integrada
- ☒ **Contexto Expandido** - Suporta hasta 4K+ tokens
- ☒ **Sem Autenticação** - Funciona sem credenciais

Limitações do LLaVA:

- ⚠ Requer hardware local (VRAM suficiente)
- ⚠ Latência maior que API remota (~2-5s para 7B)
- ⚠ Qualidade inferior ao GPT-4V (mas aceitável)
- ⚠ Requer instalação do Ollama
- ⚠ Modelos maiores (34B) precisam 24GB+ VRAM

Instalação do Ollama + LLaVA:

```
# Instalar Ollama
# Windows: https://ollama.com/download/windows
# macOS: https://ollama.com/download/mac
# Linux: curl -fsSL https://ollama.ai/install.sh | sh

# Fazer pull do modelo LLaVA
ollama pull llava:7b          # Rápido, 4GB
# ou
ollama pull llava:13b         # Mais preciso, 8GB
# ou
ollama pull llava:34b         # Mais poderoso, 24GB

# Iniciar serviço Ollama (roda em localhost:11434)
ollama serve
```

Configuração (config/config.json):

```
{
  "llm": {
    "api": "ollama",
    "model": "llava:7b",
    "base_url": "http://localhost:11434/api",
    "temperature": 0.7,
    "context_window": 2048,
    "timeout": 60
  }
}
```

3 OpenAI GPT-4V - ALTERNATIVA PREMIUM

- ✂ Status: Suportado via integração
- 🔒 Modelo: GPT-4V (Vision)
- 🌐 API: <https://api.openai.com/v1>
- 💰 Custo: \$0.01 - \$0.03 / 1K tokens (~R\$ 0.05 - R\$ 0.15)
- ⚙ Requisitos: API Key OpenAI

Vantagens:

- ☒ Qualidade superior de resposta
- ☒ Visão computacional mais precisa
- ☒ Melhor compreensão de contexto
- ☒ Suporte multilíngue excelente

Desvantagens:

- ☒ Custo recorrente por uso
- ☒ Requer conexão com internet
- ☒ Limites de taxa (rate limiting)
- ☒ Dados enviados para servidor remoto

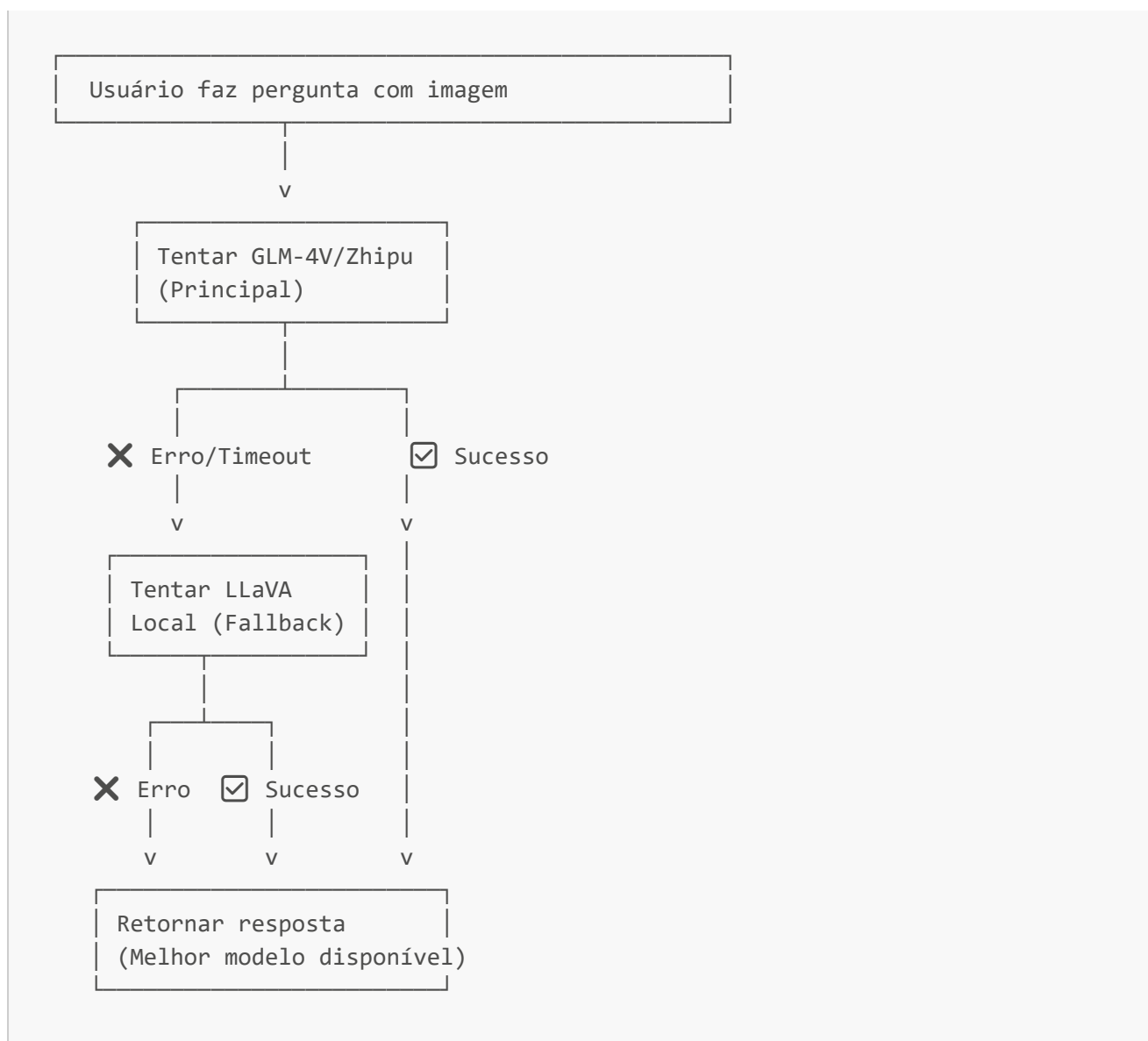
Configuração:

```
{
  "llm": {
    "api": "openai",
    "model": "gpt-4-vision-preview",
    "api_key": "sk-...",
    "base_url": "https://api.openai.com/v1"
  }
}
```

 Comparação de Modelos

Aspecto	GLM-4V (Zhipu)	LLaVA (Ollama)	GPT-4V (OpenAI)
Custo	¥0.1-0.5/1K	 Gratuito	\$0.01-0.03/1K
Latência	~500ms	~2-5s	~1-2s
Privacidade	☒ Servidor remoto	☒ Local	☒ Remoto
Internet	☒ Obrigatório	☒ Offline	☒ Obrigatório
Qualidade	★★★★	★★★	★★★★★
Customizável	☒ Não	☒ Sim	☒ Não
Rate Limit	Sim	☒ Não	Sim
Visão Comp.	☒ Sim	☒ Sim	☒ Sim
Autenticação	☒ Token	☒ Não	☒ API Key

 Estratégia de Fallback Automático



💡 Recomendação de Uso

Para Desenvolvimento/Testes: 🏠 LLaVA + Ollama Local

- Gratuito, sem limite de uso
- Execução offline garantida
- Ideal para prototipagem

Para Produção Crítica: 🛡️ GPT-4V OpenAI

- Melhor qualidade garantida
- Suporte profissional disponível
- Ideal para aplicações comerciais

Para Uso Corporativo Chinês: 🌐 GLM-4V Zhipu

- Otimizado para caracterização chinesa
- Menor custo em regiões chinesas
- Integração nativa com Xiaozhi

Ecossistema de Ferramentas MCP (32+ Ferramentas)

- **Controle de Sistema:** Monitoramento, gerenciamento de aplicativos, controle de volume
- **Gerenciador de Agenda:** Criar, consultar, atualizar, deletar eventos com lembretes inteligentes
- **Tarefas Agendadas:** Temporizadores com suporte a execução atrasada
- **Reprodutor de Música:** Busca online, controle de reprodução, letras, cache local
- **Consulta de Passagens Aéreas:** Integração com serviços de transporte
- **Busca na Web:** Pesquisa Bing e análise de conteúdo
- **Receitas:** Banco de dados rico de receitas com recomendações
- **Mapas:** Serviços de mapeamento com geocodificação, planejamento de rotas, busca por proximidade
- **Ba Zi (Astrologia Chinesa):** Análise de compatibilidade matrimonial e calendário lunar
- **Câmera:** Captura de imagens e análise com IA

Integração IoT

- **Gerenciamento Unificado:** Arquitetura baseada em padrão Thing para controle de dispositivos
- **Controle de Casa Inteligente:** Luzes, volume, sensores de temperatura
- **Sincronização de Estado em Tempo Real:** Monitoramento contínuo com atualizações incrementais
- **Design Extensível:** Drivers modulares para novos dispositivos

Processamento de Áudio Avançado

- **Processamento Multinível:** Codec Opus, reamostragem em tempo real
- **Deteção de Atividade de Fala (VAD):** Interrupção inteligente de áudio
- **Deteção de Palavra-Chave:** Modelos Sherpa-ONNX com suporte a pinyin
- **Gerenciamento de Stream:** Entradas/saídas independentes com recuperação de erros
- **Cancelamento de Eco:** Integração com módulo de processamento de áudio WebRTC
- **Gravação de Áudio do Sistema:** Captura de áudio do sistema operacional

Interface do Usuário

- **Interface Gráfica:** GUI moderna baseada em PyQt5 com expressões e exibição de texto
- **Modo Linha de Comando:** CLI para ambientes sem GUI
- **Bandeja do Sistema:** Suporte para execução em background
- **Atalhos Globais:** Teclas de atalho personalizáveis
- **Painel de Configurações:** Interface completa para personalização

Segurança e Estabilidade

- **Transmissão de Áudio Criptografada:** Protocolo WSS seguro
- **Sistema de Ativação de Dispositivo:** Suporte a protocolos v1/v2
- **Recuperação de Erros:** Reconexão automática e tratamento robusto

Suporte Multiplataforma

- **Compatibilidade:** Windows 10+, macOS 10.15+, Linux

- **Protocolos:** WebSocket e MQTT
- **Modos:** GUI e CLI
- **Otimização:** Específica para cada plataforma

Início Rápido

Requisitos do Sistema

Mínimo:

- Python 3.9 - 3.12
- Windows 10+, macOS 10.15+, ou Linux
- Microfone e alto-falante
- Conexão de internet estável

Recomendado:

- 8GB+ RAM
- CPU com suporte AVX
- 2GB espaço em disco
- Áudio com taxa de amostragem de 16kHz

Instalação

```
# Clonar repositório
git clone https://github.com/MarceloClaro/ASSISTENTE-SHI.git
cd ASSISTENTE-SHI

# Criar ambiente virtual
python -m venv venv
source venv/bin/activate # No Windows: venv\Scripts\activate

# Instalar dependências
pip install -r requirements.txt

# Executar aplicação (GUI)
python main.py --mode gui --protocol websocket

# Ou modo CLI
python main.py --mode cli
```

Documentação e Guias

- [Guia Completo VSCode \(PT\)](#) - Configuração detalhada do VSCode
- [Documentação Técnica](#) - Arquitetura e desenvolvimento
- [FAQ - Perguntas Frequentes](#) - Dúvidas comuns

Arquitetura Técnica

Design de Arquitetura

- **Arquitetura Orientada por Eventos:** Loop assíncrono asyncio com processamento de alta concorrência
- **Design em Camadas:** Separação clara entre aplicação, protocolo, dispositivos e UI
- **Padrão Singleton:** Componentes principais com gerenciamento centralizado de recursos
- **Arquitetura Plugável:** Sistema MCP e dispositivos IoT extensíveis

Componentes Principais

- **Processamento de Áudio:** Opus, WebRTC AEC, reamostragem, gravação do sistema
- **Reconhecimento de Fala:** Sherpa-ONNX offline, VAD, detecção de palavra-chave
- **Protocolos:** WebSocket/MQTT dual, transmissão criptografada, reconexão automática
- **Sistema de Configuração:** Configuração em camadas, acesso por pontos, JSON/YAML

Otimizações de Desempenho

- **Assincronismo Completo:** Sem operações bloqueantes
- **Gerenciamento de Memória:** Cache inteligente e coleta de lixo
- **Áudio de Baixa Latência:** Processamento em 5ms com gerenciamento de fila
- **Controle de Concorrência:** Pool de tarefas, semáforos, segurança de thread

Estrutura do Projeto

```
ASSISTENTE-SHI/  
├── main.py                # Ponto de entrada  
├── src/  
│   ├── application.py    # Lógica central  
│   ├── audio_codecs/     # Codificação de áudio  
│   ├── audio_processing/ # Processamento de áudio  
│   ├── core/             # Componentes principais  
│   ├── display/          # Camada abstrata de interface  
│   ├── iot/              # Gerenciamento IoT  
│   ├── mcp/              # Sistema de ferramentas MCP  
│   ├── protocols/        # Protocolos de comunicação  
│   ├── utils/            # Funções utilitárias  
│   └── views/            # Componentes de UI  
├── config/               # Arquivos de configuração  
├── models/               # Modelos de IA  
├── logs/                 # Arquivos de log  
└── requirements.txt      # Dependências Python
```

Fluxos e Diagramas do Projeto

1 Fluxo de Estados da Aplicação

INICIALIZAÇÃO DO SISTEMA

|
v

```

+-----+-----+
|  CARREGANDO  |
| Configurações |
+-----+-----+
    
```

|

v

```

+-----+-----+
|  ATIVANDO    |
| Dispositivo  |
+-----+-----+
    
```

|

v

MÁQUINA DE ESTADOS PRINCIPAL

(Palavra-chave) ou (Botão GUI)

|

v

```

+-----+-----+
|  OCIOSO  |----->|  CONECTANDO  |
+-----+-----+
    
```

^

Encerrou
Áudio

|

v

```

+-----+-----+
| ESCUTANDO / AGUARD. |
+-----+-----+
    
```

|

(Áudio detectado)
VAD ativado

|

v

```

+-----+-----+
| PROCESSANDO ÁUDIO   |
| (STT em andamento) |
+-----+-----+
    
```

|

(STT completo)

v

```

+-----+-----+
| CONVERSANDO / LLM   |
| (Processando IA)    |
+-----+-----+
    
```

Fim

|

(Resposta pronta)

|

v

```

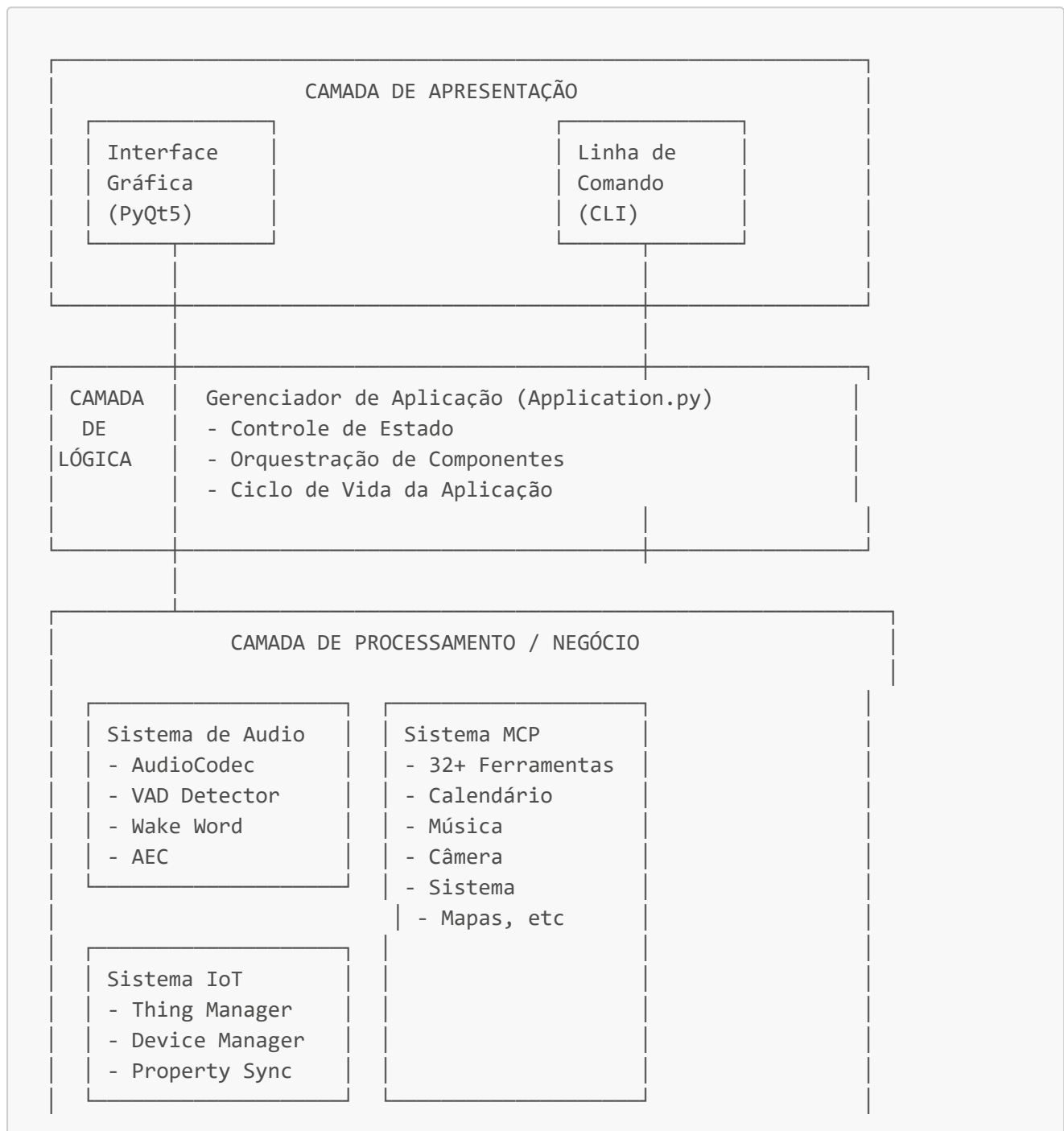
+-----+-----+
| REPRODUZINDO ÁUDIO |
+-----+-----+
    
```

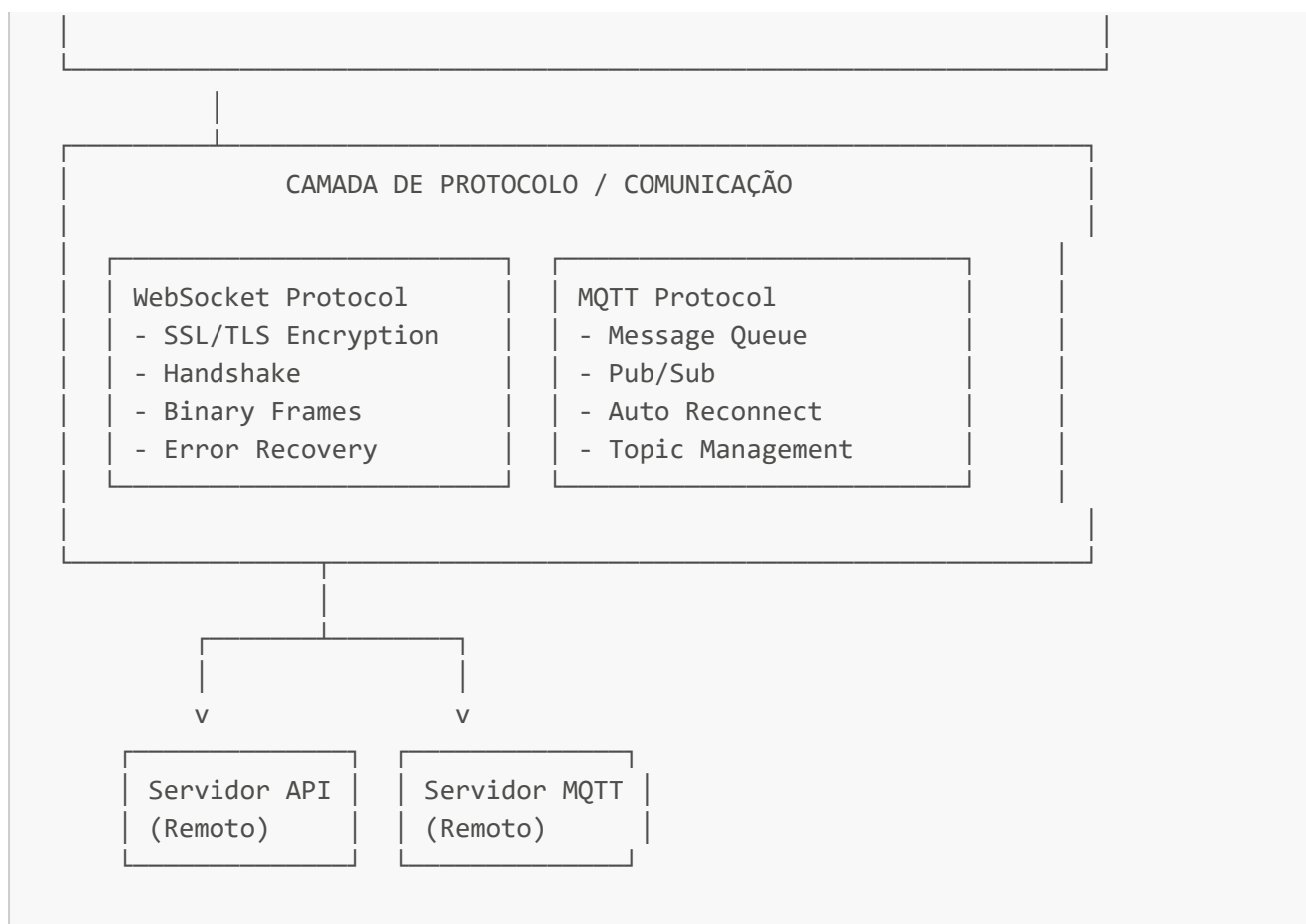
```

| (TTS em andamento) |
+-----+-----+
|
| (Reprodução finalizada)
|
v
+-----+-----+
| Retorna ESCUTANDO |
| (próxima iteração) |
+-----+-----+

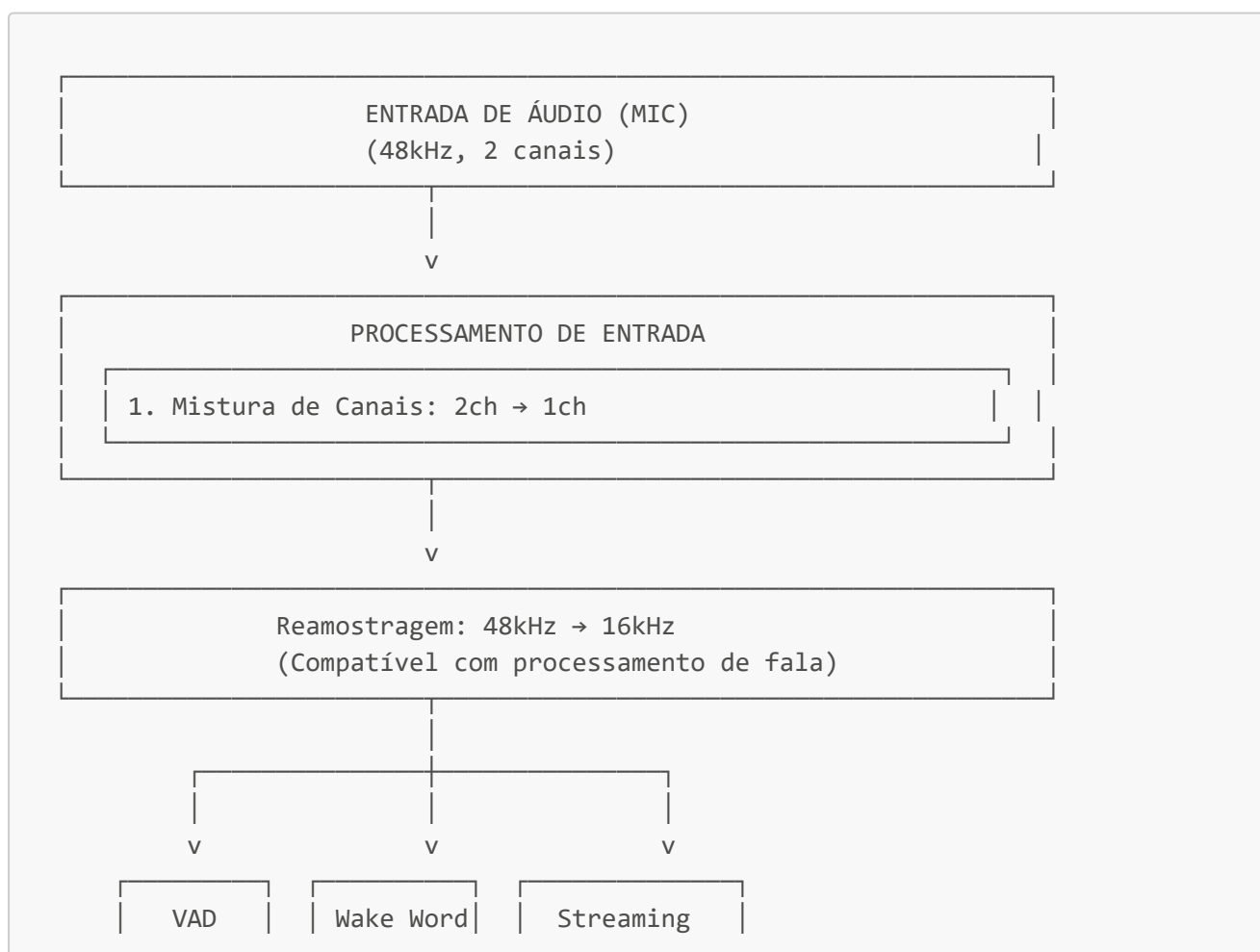
```

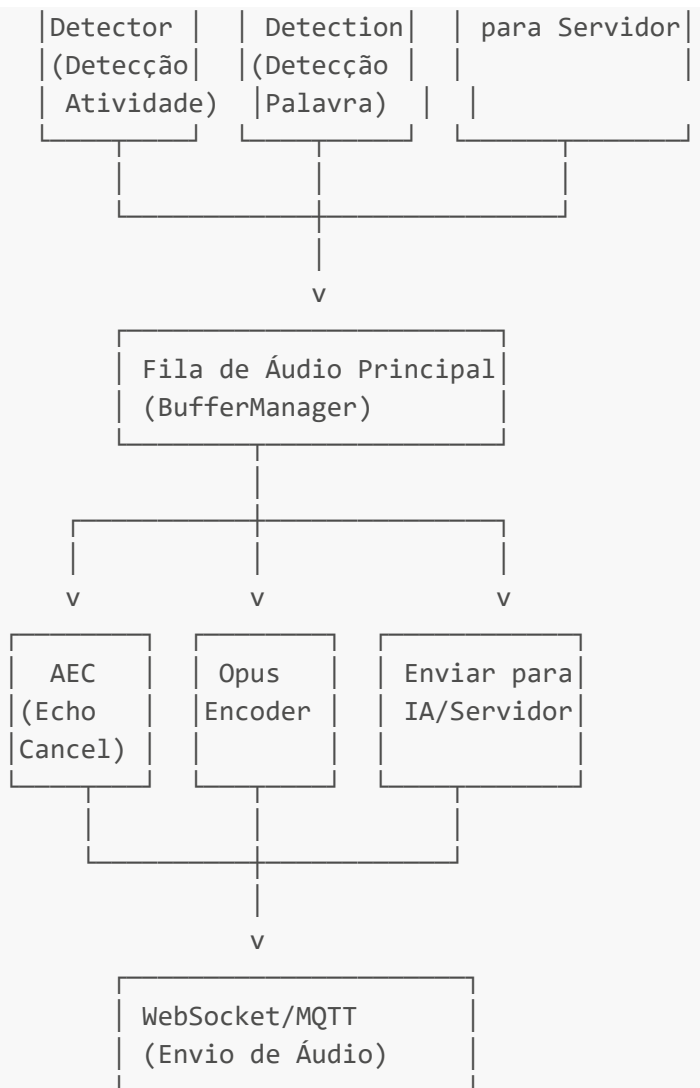
2) Arquitetura em Camadas do Sistema



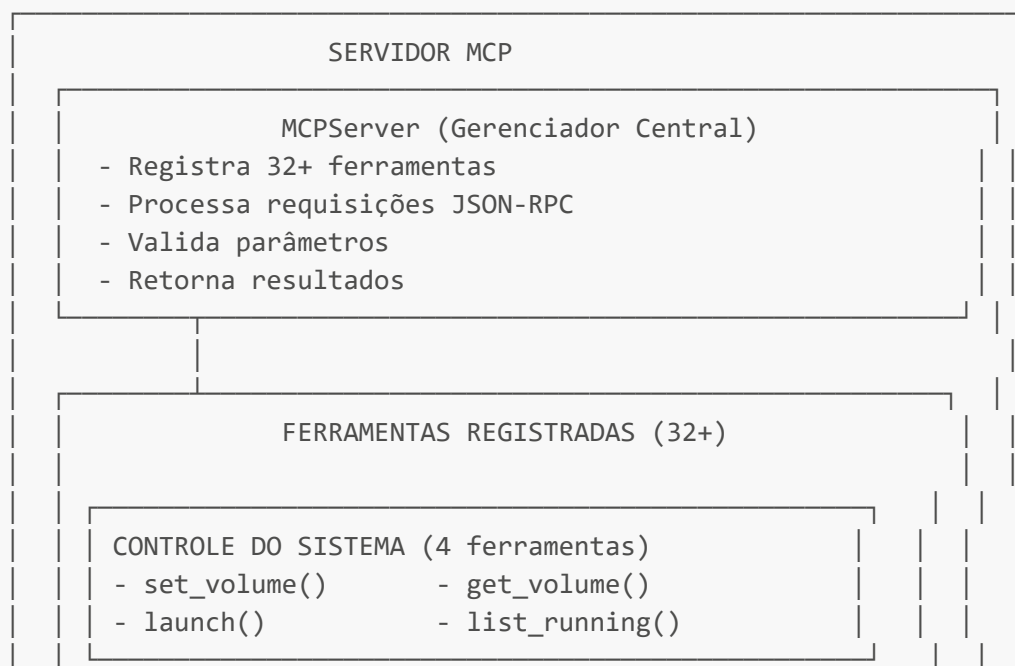


3 Fluxo de Processamento de Áudio





4 Arquitetura do Sistema MCP (Model Context Protocol)



GERENCIADOR DE CALENDÁRIO (7 ferramentas)

- create_event()
- get_events()
- update_event()
- delete_event()
- get_upcoming_events()
- get_categories()

REPRODUTOR DE MÚSICA (7 ferramentas)

- search_and_play()
- pause()
- resume()
- stop()
- seek()
- get_lyrics()
- get_local_playlist()

GERENCIADOR DE TEMPORIZADORES (3 ferramentas)

- start_countdown()
- cancel_countdown()
- get_active_timers()

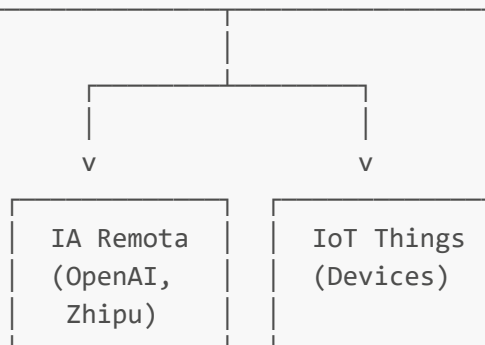
VISÃO COMPUTACIONAL (2 ferramentas)

- take_photo()
- take_screenshot()

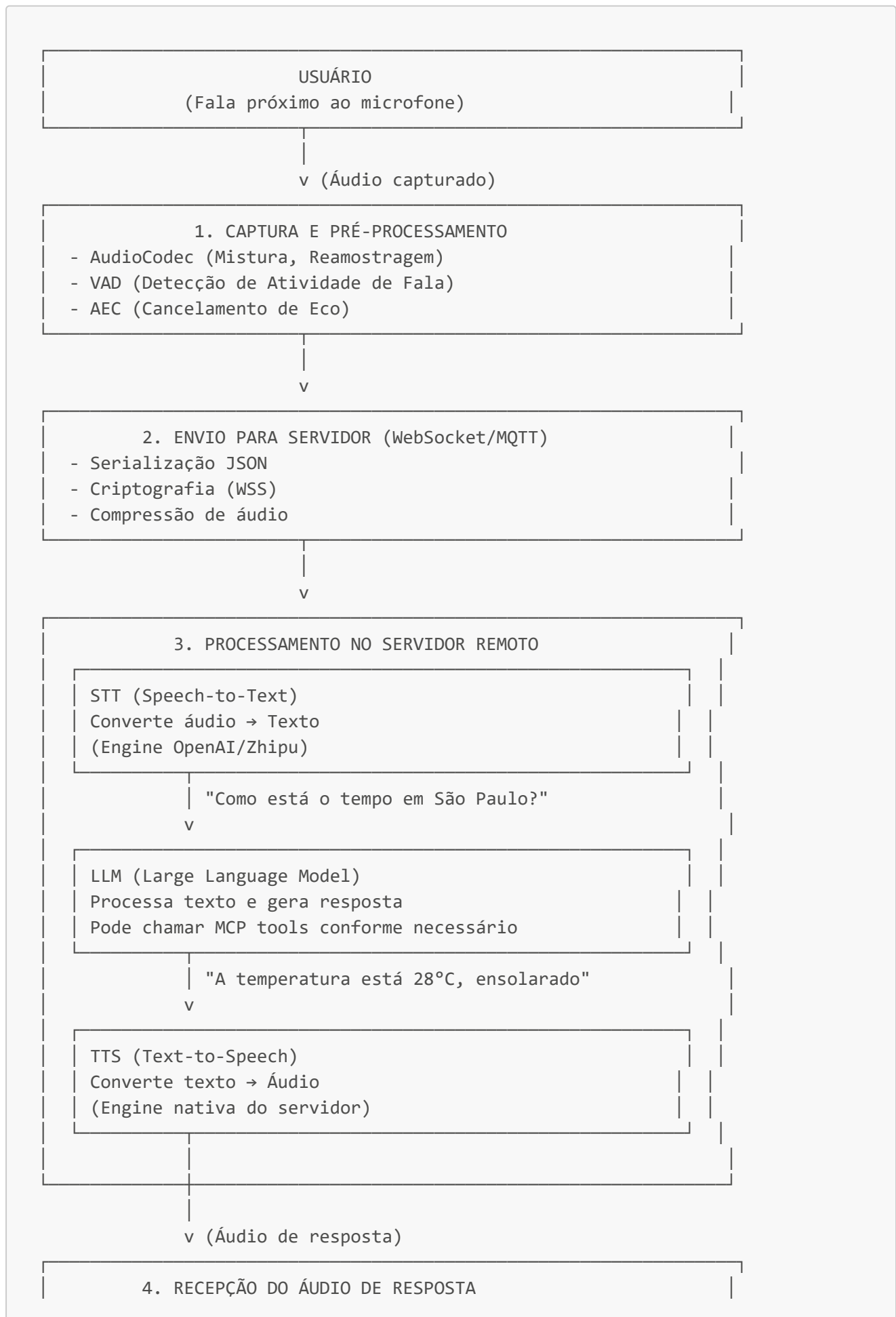
ASTROLOGIA BA ZI (6 ferramentas)

- get_bazi_detail()
- analyze_marriage_compatibility()
- get_chinese_calendar()
- build_bazi_from_lunar_datetime()

+ 3 ferramentas adicionais em desenvolvimento...



5 Fluxo Completo de Interação Usuário → IA → Resposta



- Desserialização JSON
- Descriptografia
- Descompressão de áudio

v

5. PROCESSAMENTO DE SAÍDA DE ÁUDIO

- AudioCodec (Reamostragem: 24kHz → 44100Hz)
- Opus Decode (se necessário)
- Normalização de volume

v

6. REPRODUÇÃO DE ÁUDIO (Speaker)

- Streaming para saída de áudio
- Sincronização com interface
- Feedback visual (animação)

v

USUÁRIO OUVI A RESPOSTA
(Sistema retorna ao estado ESCUTANDO)

6 Fluxo de Inicialização do Sistema

```
python main.py --mode gui --protocol websocket
```

v

parse_args()
(Processar argumentos CLI)

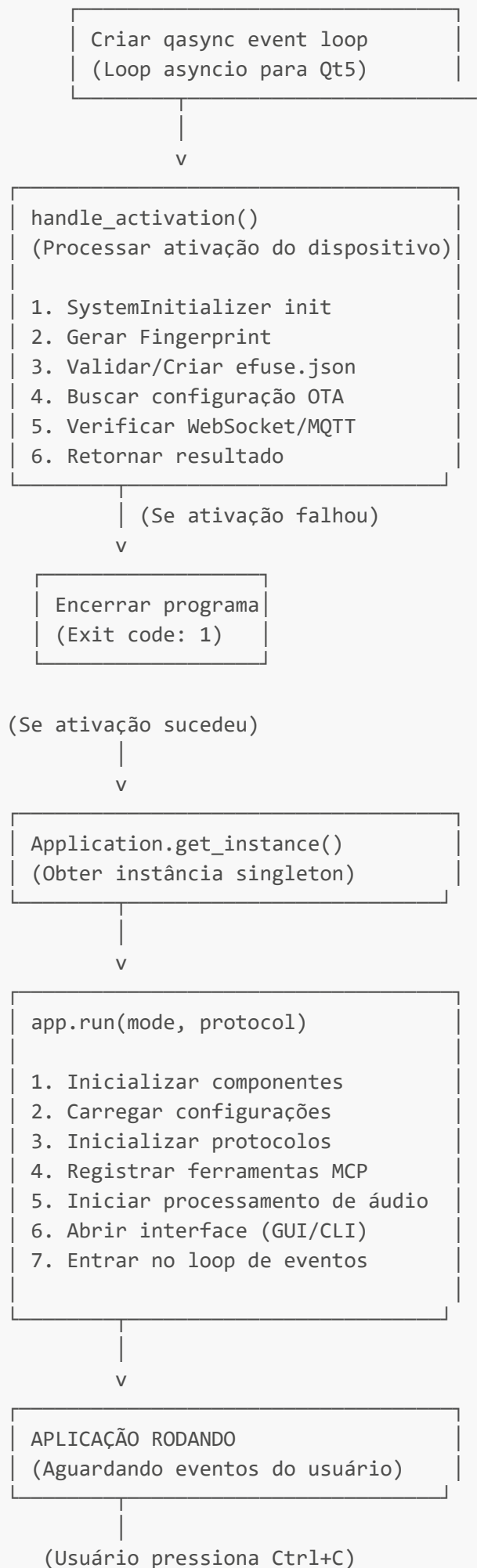
v

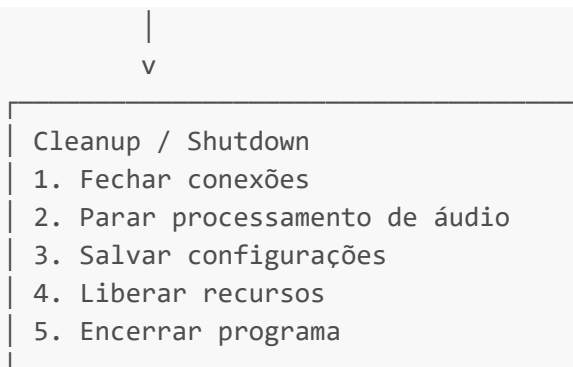
setup_logging()
(Iniciar sistema de logs)

v

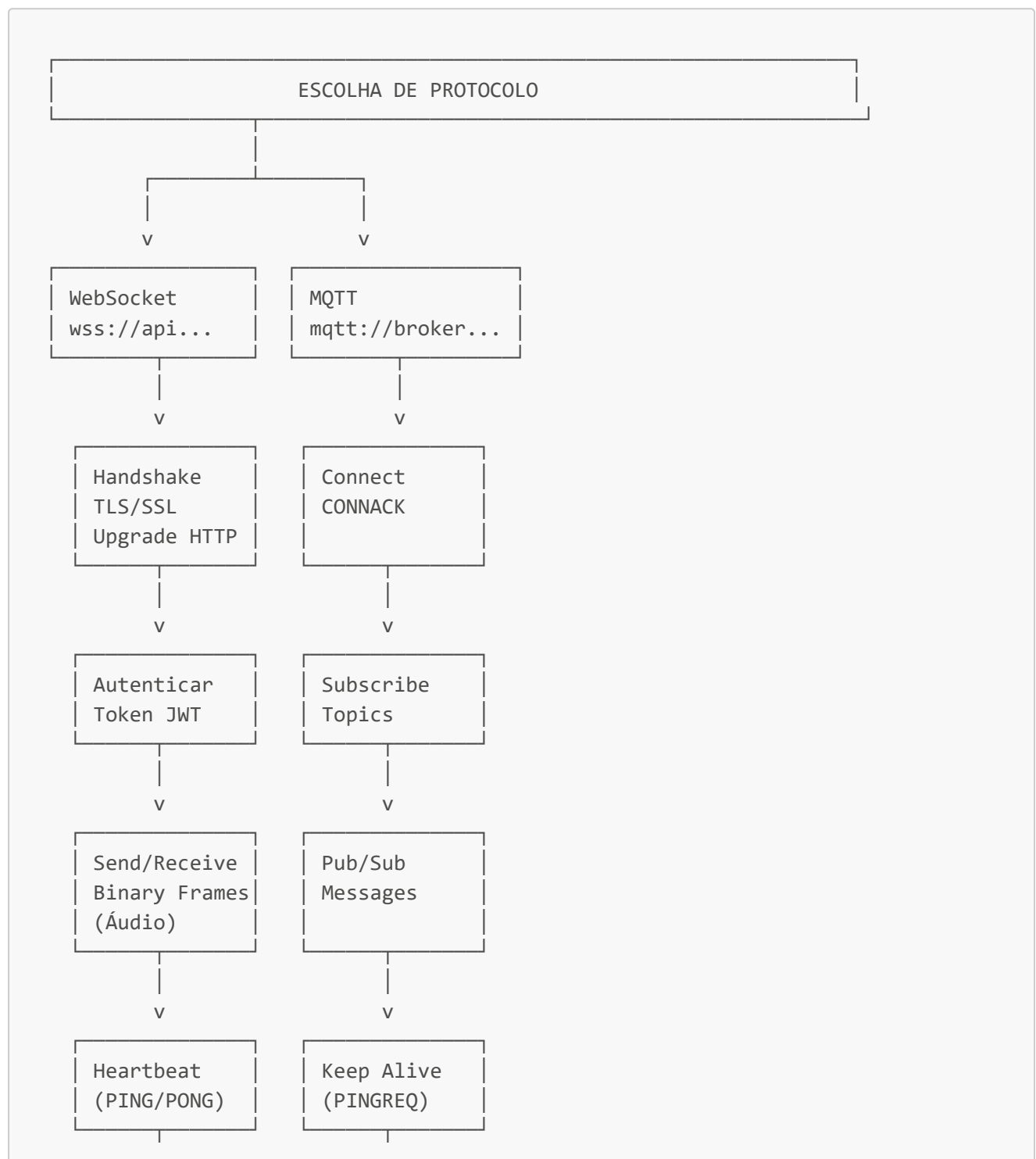
Detectar Wayland/X11
(Configurar variáveis QT)

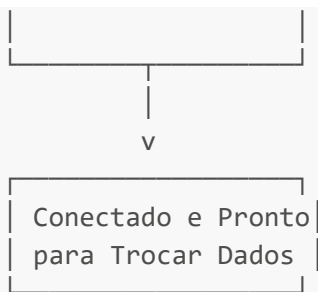
v



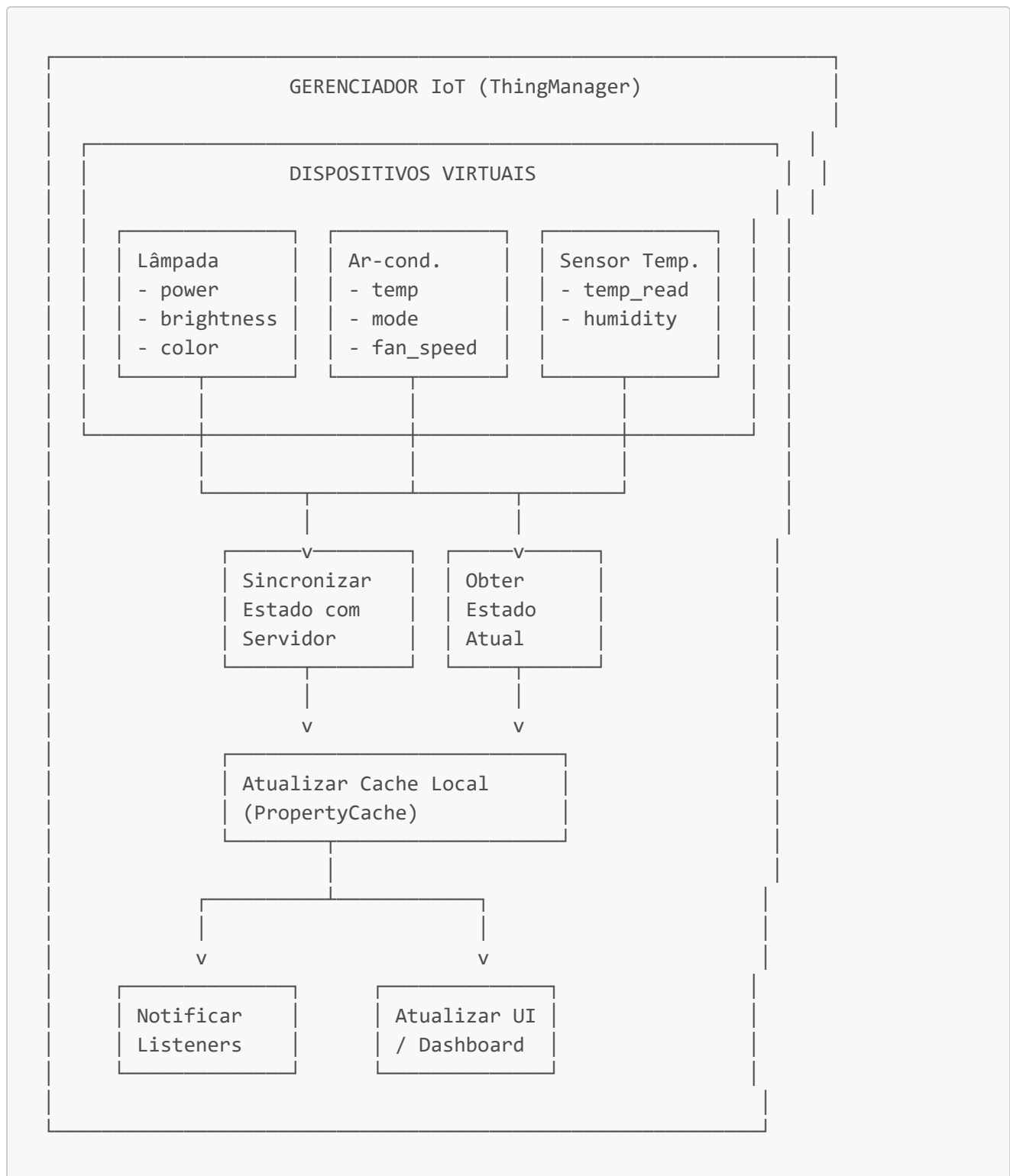


7 Fluxo de Protocolo de Comunicação





8 Fluxo de Sincronização de Estado IoT



Status Atual

☒ Funcional

- ☒ Interface GUI com WebSocket
- ☒ 32 ferramentas MCP registradas
- ☒ Processamento de áudio Opus
- ☒ Interação com IA em tempo real
- ☒ Suporte a câmera (captura de fotos)
- ☒ Gerenciamento de calendário
- ☒ Sistema de lembretes

Conhecido

-  Modelo de Wake Word ausente (Sherpa-ONNX)
-  API de Visão Remota retornando 404
-  Timeout de conexão WebSocket após ~1 minuto em teste

Desenvolvimento

Executar em Diferentes Modos

```
# Modo GUI com WebSocket (padrão)
python main.py --mode gui --protocol websocket

# Modo CLI com MQTT
python main.py --mode cli --protocol mqtt

# Pular processo de ativação (debug)
python main.py --skip-activation
```

Adicionando Novas Ferramentas MCP

Estenda a classe em `src/mcp/tools/` e registre em `mcp_server.py`.

Adicionando Novos Dispositivos IoT

Implemente `Thing` base class em `src/iot/things/`.

Estatísticas do Projeto

- **Ferramentas MCP:** 32+ funções integradas
- **Compatibilidade:** 3 sistemas operacionais
- **Protocolos:** 2 opções (WebSocket, MQTT)
- **Modos:** 2 interfaces (GUI, CLI)

Contribuições

Contribuições são bem-vindas! Por favor:

1. Faça um fork do repositório
2. Crie uma branch para sua feature (`git checkout -b feature/AmazingFeature`)
3. Commit suas mudanças (`git commit -m 'Add some AmazingFeature'`)
4. Push para a branch (`git push origin feature/AmazingFeature`)
5. Abra um Pull Request

Licença

[MIT License](#) - Veja o arquivo LICENSE para detalhes

Agradecimentos

Agradecimentos especiais ao projeto original [py-xiaozhi](#) e todos os contribuidores.

Contato e Suporte

Para dúvidas, sugestões ou relatórios de bugs, abra uma issue no repositório GitHub.

Última atualização: 13 de janeiro de 2026

Status: ☒ Operacional em modo GUI + WebSocket

